

МИНОБРНАУКИ РОССИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«Пермский государственный национальный
исследовательский университет»

Институт компьютерных наук и
технологий

ОТЧЁТ
по индивидуальной работе №2
по дисциплине «Язык программирования Python»
Варианты заданий 4, 20

Работу выполнила
студентка группы ИТ-9 2025 1
курса

Суворова Полина Михайловна
«18» июня 2026 г.

Работу проверила
Шилина Алла Владимировна
«19» июня 2026 г.

Пермь 2026

СОДЕРЖАНИЕ

Постановка задачи	3
Задача 4	3
Задача 20	4
Алгоритм решения.....	5
2.1. Структуры данных.....	5
2.2. Задача 4: генерация кода для вычислительной машины	5
2.2.1. Общая идея.....	5
2.2.2. Пошаговое описание	5
2.2.3. Пример работы.....	7
2.3. Задача 20: дерево выражений	8
2.3.1. Общая идея.....	8
2.3.2. Этап 1: преобразование инфикс → постфикс	9
2.3.3. Этап 2: построение дерева	9
2.3.4. Этап 3: вычисление дерева	10
2.3.5. Этап 4: упрощение дерева.....	10
2.3.6. Пример работы.....	11
Тестирование.....	12
3.1. Тесты для задачи 4.....	12
3.2. Тесты для задачи 20.....	13
3.3 Вывод по тестированию.....	13
Код программы	14

Постановка задачи

Задача 4

Абстрактная вычислительная машина имеет один регистр и шесть инструкций:

LD A Помещает операнд A в регистр

ST A Помещает содержимое регистра в переменную с именем A

AD A Прибавляет значение A к регистру. Результат остается в регистре.

SB A Вычитает значение A из регистра. Результат остается в регистре.

ML A Умножает содержимое регистра на значение переменной с именем A. Результат остается в регистре.

DV A Делит содержимое регистра на значение A. Результат остается в регистре.

Дано выражение в постфиксной форме, состоящее из однобуквенных операндов и операций «+», «-», «*», «/». Написать программу вывода инструкций вычислительной машины, необходимых для вычисления выражения. Результат вычисления должен оставаться в регистре, разрешается использовать обозначения T_n для временных переменных.

Пример: для выражения ABC*+DE-/ перечень инструкций имеет вид

LD B

ML C

ST T1

LD A

AD T1

ST T1

LD D

SB E

ST T2

LD T1

DV T2

Замечание: Постфиксной формой записи выражения $a \ b$ называется запись, в которой знак операции размещен за операндами $a \ b$.

Примеры:

$$a - b \rightarrow a \ b \ -$$

$$a * b + c \rightarrow a \ b * c +$$

$$a * (b + c) \rightarrow a \ b \ c + *$$

$$a + b / c / d * e \rightarrow a \ b \ c / d / e * +$$

DV T3

Задача 20

Написать программу, которая по заданной формуле строит дерево и производит вычисления с помощью построенного дерева. Формула задана в традиционной инфиксной записи, в ней могут быть скобки, максимальная степень вложенности которых ограничивается числом 10. Аргументами могут быть целые числа и переменные, задаваемые однобуквенными именами. Допустимые операции: +, -, *, /. Унарный минус допустим. С помощью построенного дерева формулы упростить формулу, заменяя в ней все поддеревья, соответствующие формулам $(f1*f3 \pm f2*f3)$ и $(f1*f2 \pm f1*f3)$ на поддеревья, соответствующие формулам $((f1 \pm f2)*f3)$ и $(f1*(f2 \pm f3))$.

Алгоритм решения

2.1. Структуры данных

Для реализации проекта используются две собственные структуры данных, описанные в файле `structures.py`:

`Stack` — стек, работающий по принципу LIFO. Реализован на основе списка. Используется для преобразования инфиксной записи в постфиксную, для построения дерева выражений и для генерации кода абстрактной машины.

`TreeNode` — узел бинарного дерева. Содержит значение `value` и ссылки на левого и правого ребёнка `left` и `right`. Если оба ребёнка отсутствуют, узел является листом и хранит операнд (число или переменную). Если `value` содержит операцию, узел является внутренним.

2.2. Задача 4: генерация кода для вычислительной машины

2.2.1. Общая идея

Алгоритм работает с постфиксной (обратной польской) записью арифметического выражения. В постфиксной записи операнды располагаются перед операциями, поэтому вычисление естественно выполняется с помощью стека. Каждый операнд помещается в стек, а каждая операция забирает из стека два верхних элемента, выполняет действие и сохраняет результат.

2.2.2. Пошаговое описание

1. Инициализация. Создаётся пустой стек для хранения имён операндов и временных переменных, пустой список для инструкций машины, счётчик временных переменных `T1`, `T2`, `T3` и т.д.

2. Подсчёт операций. Перед основным циклом подсчитывается общее количество операций в выражении. Это нужно, чтобы определить последнюю операцию — после неё результат должен остаться в регистре, а не сохраняться во временную переменную.

3. Обработка операндов. Если текущий символ — буква (переменная), она приводится к верхнему регистру и помещается в стек.

4. Обработка операций. Если текущий символ — операция (+, -, *, /), из стека извлекаются два операнда: сначала правый, затем левый. Для них генерируются команды:

LD <левый операнд>
<команда операции> <правый операнд>

где <команда операции> выбирается по таблице соответствия: + → AD, - → SB, * → ML, / → DV.

5. Сохранение результата. Если текущая операция не является последней, результат в регистре сохраняется во временную переменную T_n командой ST T_n, и имя этой переменной помещается обратно в стек. Если операция последняя, то дополнительная команда ST не добавляется — результат остаётся в регистре.

6. Контроль ошибок. Если при извлечении операндов стек оказывается пустым, выбрасывается исключение о некорректности постфиксного выражения. Если после обработки в стеке остаётся более одного элемента, также сообщается об ошибке.

2.2.3. Пример работы

```
Задача 4: Генерация кода для вычислительной машины
Введите выражение в постфиксной форме.
Операнды - однобуквенные переменные (a-z), операции: +, -, *, /
Пример: ABC*+DE-/
Выражение: ABC*+DE-/

Сгенерированные инструкции для вычислительной машины:
LD B
ML C
ST T1
LD A
AD T1
ST T2
LD D
SB E
ST T3
LD T2
DV T3
```

Рисунок 1 Пример работы задания 4

Для выражения ABC*+DE-/ алгоритм выполняет следующее:

- Операнды A, B, C помещаются в стек: [A, B, C].
- Операция *: извлекаются C и B, генерируются команды LD B и ML C, результат сохраняется в T1. Стек: [A, T1].
- Операция +: извлекаются T1 и A, генерируются LD A и AD T1, результат сохраняется в T2. Стек: [T2].
- Операнды D, E помещаются в стек: [T2, D, E].
- Операция -: извлекаются E и D, генерируются LD D и SB E, результат сохраняется в T3. Стек: [T2, T3].
- Операция / (последняя): извлекаются T3 и T2, генерируются LD T2 и DV T3. Дополнительная команда ST не добавляется.

Итоговый список инструкций:

LD	B
ML	C
ST	T1
LD	A
AD	T1
ST	T2
LD	D
SB	E
ST	T3
LD	T2
DV	T3

Операция	Команда
+	AD
-	SB
*	ML
/	DV

2.3. Задача 20: дерево выражений

2.3.1. Общая идея

Решение задачи 20 состоит в преобразовании инфиксного выражения (привычной записи с операциями между операндами и скобками) в постфиксную запись, построении по ней бинарного дерева выражений, вычислении значения дерева при заданных переменных и упрощении дерева по основным алгебраическим тождествам.

2.3.2. Этап 1: преобразование инфикс → постфикс

Для преобразования используется алгоритм сортировочной станции (shunting-yard). Выражение просматривается слева направо, при этом операнды сразу добавляются в выходную последовательность, а операции и скобки помещаются в стек.

Правила работы стека операций:

- Операнд (число или переменная) — сразу в выходную строку.
 - Открывающая скобка '(' — помещается в стек.
 - Закрывающая скобка ')' — из стека в выходную строку выталкиваются все операции до ближайшей '('.
 - Операция (+, −, *, /) — выталкивает из стека все операции с приоритетом не ниже текущей, после чего сама помещается в стек.
- Приоритеты: * и / выше, чем + и −.
- Унарный минус обрабатывается отдельно и кодируется токеном 'u-' с наивысшим приоритетом.
 - В конце все оставшиеся в стеке операции переносятся в выходную строку.

2.3.3. Этап 2: построение дерева

Постфиксная запись удобна тем, что позволяет строить дерево за один проход с помощью стека узлов. Алгоритм:

1. Просматриваем токены постфиксной записи слева направо.
2. Если токен — операнд, создаём листовой узел `TreeNode` и помещаем его в стек.
3. Если токен — бинарная операция, извлекаем два верхних узла (правый и левый), создаём новый внутренний узел с этой операцией и помещаем его обратно в стек.

4. Если токен — унарный минус 'u-', извлекаем один узел, создаём узел с операцией 'u-' и правым ребёнком-операндом.

5. В конце в стеке должен остаться ровно один узел — корень дерева выражения.

2.3.4. Этап 3: вычисление дерева

Вычисление выполняется рекурсивным обходом дерева. Если узел является листом, возвращается его значение: для числа — само число, для переменной — значение из переданного словаря переменных.

Для внутреннего узла сначала вычисляются левое и правое поддеревья, затем применяется операция, хранящаяся в узле. Узел с унарным минусом возвращает противоположное значение правого поддерева.

2.3.5. Этап 4: упрощение дерева

Упрощение выполняется рекурсивно снизу вверх. Сначала упрощаются левое и правое поддеревья, затем проверяется, можно ли упростить текущий узел по одному из базовых алгебраических правил:

- $0 + x = x$
- $x + 0 = x$
- $x - 0 = x$
- $0 - x = -x$ (заменяется на унарный минус)
- $0 * x = 0$
- $1 * x = x$
- $x * 1 = x$
- $x / 1 = x$
- $0 / x = 0$

Если ни одно правило не подходит, возвращается узел с упрощёнными детьми.

2.3.6. Пример работы

```
Задача 20: Дерево выражений
Введите выражение в инфиксной форме.
Операнды: целые числа и однобуквенные переменные.
Операции: +, -, *, /, скобки, унарный минус.
Пример: (a + b) * c
Выражение: (a + b) * c - 5

Постфиксная запись: a b + c * 5 -
Дерево выражения: (((a + b) * c) - 5)

Введите значения переменных через запятую (например, a=2,b=3), или оставьте пустым, чтобы пропустить вычисление: a=2, b=3,c=4
Результат вычисления: 15.0

Упрощённое выражение: (((a + b) * c) - 5)
Упрощённое дерево: (((a + b) * c) - 5)
```

Рисунок 2 Пример работы задания 20

Входное выражение: $(a + b) * c - 5$.

1. Преобразование в постфикс: $a b + c * 5 -$.

2. Дерево: $(((a + b) * c) - 5)$.

3. При $a = 2, b = 3, c = 4$ результат вычисления равен 15.0.

4. Упрощение не применяется, так как дерево не содержит тривиальных подвыражений.

Тестирование

Тестирование проводилось вручную путём запуска программы и проверки результатов на различных наборах входных данных.

3.1. Тесты для задачи 4

Входные данные и ожидаемые результаты:

Входное выражение	Ожидаемый результат		Статус
ABC*+DE-/	LD	B	пройден
	ML	C	
	ST	T1	
	LD	A	
	AD	T1	
	ST	T2	
	LD	D	
	SB	E	
	ST	T3	
	LD	T2	
	DV	T3	
ab+	LD	A	пройден
	AD	B	
xy*	LD	X	пройден
	ML	Y	
ab+c*	LD	A	пройден
	AD	B	
	ST	T1	

	LD	T1	
	ML	C	

3.2. Тесты для задачи 20

Входные данные и ожидаемые результаты:

Инфиксное выражение	Постфиксная запись	Результат вычисления	Упрощённое выражение
$a + b$	$a b +$	$a + b$ при $a=2, b=3$ $\rightarrow 5$	$a + b$
$a + b * c$	$a b c * +$	$1 + 2 * 3 = 7$	$a + (b * c)$
$(a + b) * c$	$a b + c *$	$(1+2) * 3 = 9$	$(a + b) * c$
$-a + 5$	$a u- 5 +$	при $a=2 \rightarrow 3$	$(-a) + 5$
$0 + x * 1$	$0 x 1 * +$	не требуется	x

3.3 Вывод по тестированию

Программа протестирована вручную на различных входных данных. Для задачи 4 проверены корректные постфиксные выражения ($ABC+DE-/$, $ab+$, xy , $ab+c*$) и некорректный ввод ($a+$). Для задачи 20 проверены выражения с разными приоритетами операций, скобками, унарным минусом и упрощением ($a + b * c$, $(a + b) * c$, $-a + 5$, $3 * (-b)$, $0 + x * 1 \rightarrow x$). Все тесты пройдены корректно. Программа корректно обрабатывает ошибки ввода и предлагает повторный ввод.

Код программы

Ссылка на код программы: <https://github.com/qwqwwwqwqq/Python->